Módulo 6 - Testando e Automatizando Testes em Aplicações Web – Cypress

Nome: Mariana Caetano Costa
Email: mariana.cae.costa@gmail.com
Telefone: (31) 9.8859-0581

---

1. PREPARAÇÃO DO AMBIENTE

Durante o módulo, o ambiente utilizado para os testes foi composto pelos seguintes recursos:

Banco API

• Git Bash: acesso à pasta **banco-api**
• Inicialização do projeto pelo comando: **npm run rest-api**
• Acesso à aplicação pelo navegador.

Banco Web

• Git Bash: acesso à pasta **banco-web**
• Inicialização do servidor pelo comando: **npm run server**
• Acesso ao sistema pelo navegador.

Banco de Dados MySQL

• Banco de dados utilizado: banco.
• Usuário: julio.lima (acesso total)
• Usuário: junior.lima (acesso limitado)

---

1. Testes de aplicações web

Testar uma aplicação web significa verificar se ela funciona corretamente em diferentes navegadores, dispositivos, resoluções e condições de uso.

Principais aspectos a testar

- Funcionalidade: botões, formulários, login, transferências e regras de negócio.
- Compatibilidade: funcionamento em diferentes navegadores e sistemas.
- Responsividade: adaptação a diferentes tamanhos de tela.
- Usabilidade: facilidade de uso e clareza das mensagens.
- Acessibilidade: possibilidade de utilização por pessoas com diferentes necessidades.
- Desempenho: velocidade de carregamento e resposta.
- Segurança: autenticação, sessões e proteção de informações.
- Integração: comunicação entre interface, API e banco de dados.

O QA deve priorizar os riscos e pensar nos diferentes caminhos que o usuário pode seguir, incluindo situações inválidas e inesperadas.

2. Testes de acessibilidade
Acessibilidade é a capacidade de uma aplicação ser utilizada por pessoas com diferentes habilidades e necessidades.

Uma ferramenta útil é a extensão Axe DevTools para Chrome, que permite realizar verificações automatizadas.

Pontos importantes

- Contraste entre texto e fundo.
- Campos de formulário com rótulos acessíveis.
- Estrutura semântica do HTML.
- Identificação e classificação de problemas.
- Registro de defeitos encontrados.

Exemplo: um contraste de 2.67:1 pode ser insuficiente quando o critério aplicável exige 4.50:1.

A ferramenta ajuda a encontrar problemas, mas não substitui os testes manuais de acessibilidade. Nem todas as barreiras podem ser detectadas automaticamente.

3. Testes de desempenho com Lighthouse
O Lighthouse, disponível no Chrome DevTools, avalia aspectos como desempenho, acessibilidade e boas práticas.

Métricas importantes:

Métrica	**O que representa**
FCP	Tempo até aparecer o primeiro conteúdo
LCP	Tempo até o maior elemento de conteúdo ficar visível
TBT	Tempo total de bloqueio da thread principal durante o período medido
CLS	Instabilidade visual da página
Speed Index	Velocidade com que o conteúdo visível é apresentado

É possível analisar diagnósticos, oportunidades de melhoria e a sequência de carregamento da página. Atenção: os resultados variam conforme o ambiente, o equipamento e as condições de execução. O erro NO_FCP indica que o Lighthouse não detectou o primeiro conteúdo durante a medição; verifique se a página carregou e execute novamente.

4. Testes de responsividade
Utilize o modo de dispositivos do Chrome DevTools para verificar o comportamento da interface em diferentes larguras.

Por exemplo, se uma aplicação muda de layout no breakpoint de 992px, teste as larguras de 992px e 993px, além de outros tamanhos relevantes.

Verifique:

- Elementos cortados ou sobrepostos.
- Textos e botões fora da tela.
- Menus e formulários.
- Alterações de layout.
- Funcionamento das ações em dispositivos menores.

5. Testes isolados com mocks

Um mock simula o comportamento de um componente externo, como uma API.

Ferramentas como Mockoon permitem criar respostas simuladas para testar o frontend mesmo quando o backend ainda não está pronto.

Exemplo de fluxo:

1. A interface envia uma requisição POST /login.
2. O mock recebe os dados de login.
3. O mock devolve uma resposta simulada, como um token.
4. O QA verifica como a interface reage à resposta.

O contrato da API — endpoint, método, dados e formato da resposta — deve corresponder ao esperado pela aplicação.

6. Erros comuns em aplicações web

Durante os testes, procure identificar:

- Validações incorretas de formulários.
- Links e botões que não funcionam.
- Fluxos de usuário interrompidos.
- Cálculos incorretos.
- Mensagens de erro inadequadas.
- Problemas de sessão e logout.
- Informações técnicas expostas ao usuário.
- Problemas de compatibilidade, desempenho e acessibilidade.

Um bom relato de defeito deve incluir título objetivo, ambiente, pré-condições, passos para reproduzir, resultado esperado, resultado obtido e evidências.

7. Introdução ao Cypress

O Cypress é uma ferramenta de automação de testes para aplicações web. Permite interagir com a interface e verificar os resultados.

Estrutura básica de um teste

```
describe('Login', () => {
  it('deve permitir o acesso com dados válidos', () => {
    cy.visit('http://localhost:4000');
```

```
    cy.get('#username').type('Julio.Lima');
    cy.get('#password').type('23456');
    cy.contains('button', 'Entrar').click();

    cy.contains('h4', 'Realizar transferência')
      .should('be.visible');
  });
});
```

Principais comandos:

Comando	**Finalidade**
describe()	Agrupa testes relacionados
it()	Define um caso de teste
cy.visit()	Abre uma página
cy.get()	Localiza elementos por seletor
cy.contains()	Localiza elementos pelo texto
.click()	Clica em um elemento
.type()	Digita um valor
.clear()	Limpa um campo
.should()	Verifica uma condição
.select()	Seleciona uma opção em um <select> nativo

Os seletores, os dados e as mensagens dos exemplos devem corresponder à aplicação real.

8. Preparação do ambiente e do repositório

O projeto da aula utiliza uma API, uma aplicação web e um repositório de testes.

Conforme a estrutura apresentada:

- API: pasta banco-api.
- Aplicação web: pasta banco-web.
- Testes Cypress: repositório de automação separado.

A API e a aplicação web precisam estar em execução antes dos testes que dependem delas.

Comandos usados na preparação de um projeto:

```
npm init -y
npm install --save-dev cypress
npx cypress open
```

O npm init -y cria o package.json; a instalação adiciona o Cypress como dependência de desenvolvimento; e npx cypress open abre a interface para configurar e executar testes.

Estrutura típica:

```
projeto/
 ├── cypress/
|   ├── e2e/
|   ├── fixtures/
|   └── support/
 ├── cypress.config.js
 ├── package.json
└── package-lock.json
```

- e2e/: testes automatizados.
- fixtures/: arquivos com dados de teste.
- support/: configurações e comandos reutilizáveis.
- cypress.config.js: configuração do Cypress.
- package.json: dependências e scripts.

9. Testes de login válidos e inválidos

Além de executar ações, o teste precisa verificar o resultado esperado.

Exemplo de login inválido:

```
it('deve apresentar erro com dados inválidos', () => {
  cy.visit('http://localhost:4000');

  cy.get('#username').type('Julio.Lima');
  cy.get('#password').type('65432');
  cy.contains('button', 'Entrar').click();

  cy.get('.toast')
    .should('be.visible')
    .and('have.text', 'Erro no login, tente novamente');
});
```

should('be.visible') verifica a visibilidade do elemento. Já and('have.text', ...) verifica se o texto corresponde exatamente ao informado.

Se a aplicação apresentar textos diferentes, a asserção precisa ser adaptada.

O Cypress também oferece recursos para acompanhar a execução, inspecionar elementos e investigar falhas. it.only() permite executar temporariamente um teste específico; remova-o antes de executar a suíte completa.

10. Versionamento com Git

O Git permite registrar alterações e colaborar por meio de branches e Pull Requests.

Fluxo básico:

```
git checkout main
git pull
git checkout -b nome-da-alteracao

git status
git add .
git commit -m "Descreve a alteração"
git push -u origin nome-da-alteracao
```

Depois, abra um Pull Request no GitHub para revisão. Após o merge, atualize a branch principal:

```
git checkout main
git pull
```

Antes de executar git add ., confira o git status para evitar incluir arquivos indesejados, como evidências volumosas ou arquivos temporários.

11. Reduzindo duplicação com hooks do Mocha

Hooks permitem executar ações em momentos específicos do ciclo de testes.

Hook	Quando é executado
before()	Uma vez antes dos testes do bloco
beforeEach()	Antes de cada teste
afterEach()	Depois de cada teste
after()	Uma vez depois dos testes do bloco

Exemplo:

```
describe('Login', () => {
  beforeEach(() => {
    cy.visit('http://localhost:4000');
  });

  it('deve permitir login válido', () => {
    // Ações e validações
  });

  it('deve apresentar erro no login inválido', () => {
    // Ações e validações
  });
});
```

O beforeEach() evita repetir cy.visit() em todos os casos.

Conceito-chave: utilize hooks para organizar ações comuns, sem esconder etapas importantes dos testes.

12. Executando testes pela linha de comando

O Cypress oferece diferentes formas de execução:

Comando	**Finalidade**
npx cypress open	Abre a interface gráfica
npx cypress run	Executa os testes em modo headless
npx cypress run --headed	Executa com o navegador visível
npx cypress run --browser chrome	Solicita a execução no Chrome
npx cypress run --spec "cypress/e2e/login.cy.js"	Executa um arquivo específico

O modo headless não exibe o navegador e é comum em pipelines de CI/CD. O modo headed exibe o navegador durante a execução.

Scripts no package.json
Preserve os scripts já existentes e adicione os atalhos necessários:

```
{
  "scripts": {
    "test": "cypress run",
    "cy:headed": "cypress run --headed",
    "cy:open": "cypress open"
  }
}
```

Execução:
npm test
npm run cy:headed
npm run cy:open

Em CI/CD, os testes podem ser executados automaticamente para detectar problemas antes da publicação de uma versão.

13. Vídeos e screenshots
No cypress.config.js, é possível configurar a gravação de vídeos e a captura de screenshots em caso de falha.

```
const { defineConfig } = require('cypress');

module.exports = defineConfig({
  video: true,

  e2e: {
    screenshotOnRunFailure: true,
```

```
  setupNodeEvents(on, config) {
    // Eventos adicionais, se necessários
  },
 },
});
```

- video: true: habilita vídeos das execuções via linha de comando.
- screenshotOnRunFailure: true: habilita screenshots automáticos quando os testes falham durante cypress run.

Caminhos padrão:

- Vídeos: cypress/videos/
- Screenshots: cypress/screenshots/

Também é possível tirar uma captura manual:
`cy.screenshot('login-realizado-com-sucesso');`

Vídeos registram a execução; screenshots registram uma imagem. Ambos ajudam a investigar defeitos, mas ocupam espaço e devem ser mantidos conforme a necessidade do projeto.

14. Utilizando fixtures

Fixtures são arquivos externos que armazenam dados utilizados nos testes. Normalmente ficam em cypress/fixtures/.

Exemplo de credenciais.json:

```
{
 "valida": {
  "usuario": "Julio.Lima",
  "senha": "23456"
 },
 "invalida": {
  "usuario": "Julio.Lima",
  "senha": "65432"
 }
}
```

Uso no teste:

```
cy.fixture('credenciais').then((credenciais) => {
 cy.get('#username').type(credenciais.valida.usuario);
 cy.get('#password').type(credenciais.valida.senha);
 cy.contains('button', 'Entrar').click();
});
```

- cy.fixture('credenciais') carrega o arquivo sem precisar informar .json.

- .then() disponibiliza os dados carregados.
- credenciais.valida.usuario acessa a propriedade usuario do objeto valida.

Vantagem: os dados ficam separados do código. Se uma credencial de teste mudar, pode ser suficiente alterar a fixture.

Para evitar versionar screenshots, pode-se incluir cypress/screenshots/ no .gitignore. Se arquivos já estiverem rastreados pelo Git, será necessário removê-los do índice para que a regra passe a ser aplicada a eles.

Documentação: https://docs.cypress.io/api/commands/fixture

15. Variáveis de ambiente

Variáveis de ambiente permitem alterar parâmetros da execução sem modificar diretamente o código dos testes.
Exemplo de scripts no package.json:

```
{
  "scripts": {
    "test": "cypress run --env URL=http://localhost:4000",
    "test-qa": "cypress run --env URL=https://qa.exemplo.com",
    "test-prod": "cypress run --env URL=https://exemplo.com"
  }
}
```

Os endereços de QA e produção são exemplos e devem ser substituídos pelos endereços reais.

No teste:

```
cy.visit(Cypress.env('URL'));
```

Execução:

```
npm test
npm run test-qa
npm run test-prod
```

O valor de URL muda conforme o script executado. Isso permite reutilizar os mesmos testes em diferentes ambientes.

Atenção: testes em produção exigem autorização e cuidados para evitar alterações em dados reais.

16. Configurando baseUrl

O baseUrl define o endereço-base da aplicação no arquivo cypress.config.js.

```
const { defineConfig } = require('cypress');

module.exports = defineConfig({
  e2e: {
```

```
    baseUrl: 'http://localhost:4000',
  },
});
```

Com essa configuração:

```
cy.visit('/login');
cy.request('/usuarios');
```

Os comandos utilizam o endereço-base configurado para resolver os caminhos relativos.

Diferença entre baseUrl e variáveis de ambiente

- baseUrl: endereço-base utilizado por comandos como cy.visit() e cy.request() quando recebem caminhos relativos.
- Variáveis de ambiente: valores que variam conforme a execução, como tokens, parâmetros e configurações.

Use baseUrl para evitar repetir o início das URLs. Use variáveis de ambiente para parâmetros que precisam mudar entre execuções.

17. Localizando elementos sem ID único

Quando um elemento não possui um identificador exclusivo, é possível usar seletores CSS, textos e relações da estrutura HTML.

Exemplo:

```
cy.get('label[for="conta-origem"]')
  .parent()
  .as('campoContaOrigem');

cy.get('@campoContaOrigem').click();
```

- label[for="conta-origem"]: localiza um label pelo atributo for.
- .parent(): seleciona o elemento pai direto.
- .as('campoContaOrigem'): cria um alias.
- cy.get('@campoContaOrigem'): reutiliza o alias.

Para abrir um componente personalizado e selecionar uma opção:

```
cy.get('label[for="conta-origem"]')
  .parent()
  .click();

cy.contains('Priscila').click();
```

O exemplo depende da estrutura real do HTML. Se houver vários elementos com o mesmo texto, restrinja a busca ao componente correto.

Campo de valor e botão de transferência

```
cy.get('#valor').clear().type('11');
cy.contains('button', 'Transferir').click();
cy.contains('Transferência realizada').should('be.visible');
```

Para um elemento HTML nativo <select>, pode-se usar:

```
cy.get('select[name="conta-origem"]').select('Priscila');
```

Esse comando não se aplica automaticamente a componentes personalizados que apenas imitam um campo de seleção.

Boas práticas

- Prefira seletores estáveis, exclusivos e fáceis de entender.
- Quando possível, utilize atributos próprios para automação, como data-cy.
- Evite depender de posições e estruturas HTML frágeis.
- Confira o HTML antes de usar parent().
- Valide o resultado das ações com asserções.
- Evite que os testes dependam de dados deixados por execuções anteriores.

Exemplo de seletor próprio:

```
cy.get('[data-cy="valor-transferencia"]')
  .clear()
  .type('11');
```

18. Custom Commands

Custom Commands são comandos personalizados criados com Cypress.Commands.add(). Eles permitem reaproveitar ações e reduzir duplicação.

Organização sugerida

```
cypress/
└── support/
    ├── commands.js
    └── commands/
        ├── common.js
        ├── login.js
        └── transferencias.js
```

- common.js: comandos genéricos.
- login.js: comandos relacionados ao login.
- transferencias.js: comandos de transferência.
- commands.js: importa os módulos personalizados.

Exemplo de importações em commands.js:
import './commands/common';
import './commands/login';
import './commands/transferencias';
Os caminhos devem corresponder aos arquivos existentes no projeto.

Criando um comando

```
Cypress.Commands.add('verificarMensagemToast', mensagem => {
  cy.get('.toast').should('have.text', mensagem);
});
```

Uso:

```
cy.verificarMensagemToast(
  'Autenticação necessária para transferências acima de R$5.000,00.'
);
```

O parâmetro mensagem permite reutilizar o comando com diferentes textos.

Comando para selecionar uma opção

```
Cypress.Commands.add(
  'selecionarOpcaoNaCombobox',
  (labelDoCampo, opcao) => {
    cy.get(`label[for="${labelDoCampo}"]`)
      .parent()
      .click();

    cy.contains(opcao).click();
  }
);
```

Uso:

```
cy.selecionarOpcaoNaCombobox('conta-origem', 'Priscila');
cy.selecionarOpcaoNaCombobox('conta-destino', 'Julio');
```

A expressão entre crases é uma template string: ${labelDoCampo} insere o valor da variável na string.

Comando para realizar uma transferência

```
Cypress.Commands.add(
  'realizarTransferencia',
  (contaOrigem, contaDestino, valor) => {
    cy.selecionarOpcaoNaCombobox(
      'conta-origem',
      contaOrigem
    );

    cy.selecionarOpcaoNaCombobox(
```

```
    'conta-destino',
    contaDestino
   );

   cy.get('#valor').clear().type(String(valor));
   cy.contains('button', 'Transferir').click();
  }
);
```

Uso:

```
cy.realizarTransferencia('Priscila', 'Julio', 11);
```

Para verificar a regra de autenticação acima de R$ 5.000:

```
it('deve exigir autenticação acima de 5000', () => {
  cy.fazerLoginComCredenciaisValidas();

  cy.realizarTransferencia('Priscila', 'Julio', 6000);

  cy.verificarMensagemToast(
   'Autenticação necessária para transferências acima de R$5.000,00.'
  );
});
```

Esse exemplo pressupõe que os comandos de login e transferência estejam implementados e que os dados utilizados sejam válidos no ambiente de teste.

Importante: Custom Commands organizam ações reutilizáveis, mas não substituem as asserções. O teste ainda precisa verificar o resultado esperado.

19. Relatórios HTML com Mochawesome

O cypress-mochawesome-reporter gera relatórios HTML com informações sobre a execução dos testes.
O relatório pode apresentar:

- Quantidade de testes aprovados e reprovados.
- Tempo de execução.
- Detalhes dos erros.
- Screenshots associados aos testes, conforme a configuração.

Instalação

```
npm install --save-dev cypress-mochawesome-reporter
```

No cypress.config.js, configure o reporter:

```
reporter: "cypress-mochawesome-reporter",

reporterOptions: {
  reportDir: "cypress/reports/mochawesome",
  reportFilename: "mochawesome",
  html: true,
```

```
  json: true,
  overwrite: true,
  charts: true,
  embeddedScreenshots: true,
  saveAllAttempts: false,
  reportPageTitle: "Relatório de Testes Cypress"
},
```

Registre o plugin dentro de setupNodeEvents:
require("cypress-mochawesome-reporter/plugin")(on);
No arquivo cypress/support/e2e.js:
import 'cypress-mochawesome-reporter/register';

Execute os testes:
npm test
Com a configuração apresentada, o relatório HTML é esperado em:
cypress/reports/mochawesome/mochawesome.html

O caminho real deve ser confirmado após a execução, pois depende da configuração do projeto.

O relatório facilita a consulta dos resultados por profissionais de QA, desenvolvedores e pessoas de negócio.

Documentação: https://github.com/LironEr/cypress-mochawesome-reporter

20. Criando o README com IA generativa

O README documenta o propósito do projeto e orienta outras pessoas a instalá-lo e executá-lo.
O processo apresentado na aula é:

1. Abrir o projeto no VS Code.
2. Instalar a extensão oficial GitHub Copilot, caso disponível.
3. Abrir o chat do Copilot.
4. Solicitar a criação do README com base nos arquivos reais.
5. Revisar o resultado antes de publicá-lo.

O que documentar

- Objetivo do projeto.
- Tecnologias utilizadas.
- Estrutura de arquivos.
- Instalação e execução.
- Testes automatizados disponíveis.
- Custom Commands.
- Configuração e localização dos relatórios.
- Dependências da API e da aplicação web.

A IA deve utilizar o conteúdo real do repositório, sem inventar funcionalidades, testes, comandos ou dependências.

21. Git, testes e documentação: fluxo recomendado

Um fluxo de trabalho organizado para uma alteração nos testes é:

1. Atualizar a branch principal.
2. Criar uma branch para a alteração.
3. Implementar ou ajustar os testes.
4. Executar os testes.
5. Verificar os resultados e as evidências.
6. Conferir o git status.
7. Fazer commit e enviar a branch.
8. Abrir um Pull Request.
9. Revisar e integrar a alteração.

Exemplo de comandos:
git checkout main
git pull
git checkout -b melhoria-nos-testes

npx cypress run

git status
git add .
git commit -m "Melhora os testes automatizados"
git push -u origin melhoria-nos-testes

O exemplo usa nomes ilustrativos. Adapte o nome da branch e a mensagem do commit à alteração realizada.

22. Revisão dos conceitos fundamentais

Conceito	**O que lembrar**
Testes funcionais	Verificam se as funcionalidades atendem aos requisitos
Testes de acessibilidade	Avaliam barreiras de utilização
Lighthouse	Analisa desempenho e outros aspectos da página
Mock	Simula uma dependência externa
Cypress	Automatiza interações e verificações em aplicações web
describe()	Agrupa testes
it()	Define um caso de teste
Hooks	Executam preparações e ações em momentos específicos

beforeEach()	Executa antes de cada teste
Fixtures	Separam os dados do código dos testes
Variáveis de ambiente	Permitem alterar parâmetros entre execuções
baseUrl	Define o endereço-base da aplicação
Custom Commands	Reaproveitam ações e procedimentos
Screenshots	Registram imagens da execução
Vídeos	Registram a execução dos testes
Mochawesome	Produz relatórios HTML
README	Documenta o projeto
Git	Versiona as alterações
Pull Request	Permite revisar alterações antes da integração
CI/CD	Automatiza etapas de integração, teste e entrega

23. Checklist de aprendizado

Módulo 6 — Checklist de Aprendizado: Cypress
Marque cada item conforme conseguir realizar a atividade.

Testes de aplicações web

☐ 1. Explicar os principais tipos de testes em aplicações web.
☐ 2. Identificar problemas básicos de acessibilidade e responsividade.
☐ 3. Explicar as métricas fundamentais do Lighthouse.
☐ 4. Preparar o ambiente e executar a aplicação necessária aos testes.

Automação com Cypress

☐ 5. Criar um teste de login válido com Cypress.
☐ 6. Criar um teste de login inválido com uma asserção.
☐ 7. Utilizar beforeEach() para reduzir repetição.
☐ 8. Executar testes em modo headless e headed.
☐ 9. Configurar screenshots e vídeos.
☐ 10. Armazenar dados em fixtures.
☐ 11. Configurar e utilizar baseUrl e variáveis de ambiente.
☐ 12. Localizar elementos com seletores CSS e aliases.
☐ 13. Criar e utilizar Custom Commands.
☐ 14. Configurar e consultar um relatório HTML.

Git e documentação

☐ 15. Documentar o projeto em um README.
☐ 16. Versionar alterações com Git e abrir um Pull Request.

Avaliação final

- Total de atividades: 16
- Atividades concluídas: _____
- Atividades que preciso praticar novamente: _____
- Próxima atividade de estudo: _____

24. Referências oficiais

- Cypress: https://docs.cypress.io/
- Organização dos testes: https://docs.cypress.io/app/core-concepts/writing-and-organizing-tests
- Fixtures: https://docs.cypress.io/api/commands/fixture
- Custom Commands: https://docs.cypress.io/api/cypress-api/custom-commands
- Comando parent(): https://docs.cypress.io/api/commands/parent
- Comando contains(): https://docs.cypress.io/api/commands/contains
- Comando as(): https://docs.cypress.io/api/commands/as
- Boas práticas: https://docs.cypress.io/app/core-concepts/best-practices
- W3Schools — <select>: https://www.w3schools.com/tags/tag_select.asp
- W3Schools — Template Strings: https://www.w3schools.com/js/js_string_templates.asp

Conclusão do módulo

O objetivo é aprender a planejar testes, identificar problemas em aplicações web e automatizar cenários com Cypress. A evolução passa por escrever testes claros, verificar resultados com asserções, reduzir duplicação, organizar os dados, investigar falhas e documentar as execuções.

Para trabalhar com QA, o mais importante não é apenas saber executar comandos: é entender o comportamento esperado, escolher cenários relevantes, detectar defeitos e apresentar evidências confiáveis.